

Evolutionary Prompt Optimization

Domain Identification & Classification

Team Four

CSE 4392 / 5320



Agenda

- Overview
- Automated Prompt Evolution Method & Results
- Chain of Thought Methods & Results
- Takeaways

Overview

Objectives

What are we trying to achieve?

Objective: Optimize a prompt so that an LLM following the AUM methodology identifies and classifies domain-specific phrases **with $F1 \geq 90\%$**

- Devise consistent scoring methods to evaluate model performance.
- Evaluate autonomous prompt optimization
- Develop prompt that can generalize well to various problem domains.
- Evaluate different optimization strategies.

Tasks

Models evaluated on **identification** and **classification** tasks:

Identification: Given a description of requirements for a software application, identify domain specific phrases according to AUM rules.

Classification: Given a list of domain specific phrases, classify into classes, attributes, attribute values and other elements of a software design.

Metrics

CONFUSION MATRIX

TP = True Positives
TN = True Negatives
FP = False Positives
FN = False Negatives

	p' (Predicted)	n' (Predicted)
p (Actual)	True Positive	False Negative
n (Actual)	False Positive	True Negative

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

$$\text{F1-score} = \frac{2 * \text{precision} * \text{recall}}{\text{precision} + \text{recall}}$$

Automated Prompt Optimization Method & Results

Automated Prompt Optimization

Data cleaning and prompting

- Leverage LangGraph to create a framework for evaluating prompt optimization strategies across various domains.
- **Data Cleaning:** Converted pdf ground truths into csv files to allow for deterministic scoring. Pre-determined list of past tense and word variations counted as a match by adding to dataset.
- **Initial prompt:** Classification and Identification prompts manually written. Each prompt contains instructions to only output csv to be scored against ground truth. Prompts also contain placeholders for the injection of context such as application description and domain phrases

rental.csv	
1	rule,phrase
2	1,vehicle
3	1,manufacturer
4	1,price class
5	1,rental price
6	4,available / not available
7	4,rented out
8	4,purchase
9	4,repair
10	4,maintenance
11	4,disposal
12	1,car / passenger car
13	2,makes of car
14	2,model of car

Car rental Ground Truth
(Identification)

car_rental.csv	
1	label,element,arg1,arg2
2	C,Vehicle,,
3	A,manufacturer,,
4	A,price class,,
5	A,price,,
6	A,available,,
7	A,rented out,,
8	A,status,,
9	V,purchase,status,
10	V,repair,status,
11	V,maintenance,status,
12	V,disposed,status,
13	C,Passenger Car,,
14	A,make,,
15	A,model,,

Car rental Ground Truth
(Classification)

Task

Read the business description below and list every domain-specific phrase you can find in it.

Business Description

<DESCRIPTION>~

Rule numbers

Categorize each phrase with the matching rule number:

- 1 = nouns / noun phrases
- 2 = "X of Y" expressions
- 3 = transitive verbs
- 4 = adjectives and list items
- 5 = numbers and quantities
- 6 = possession ("has")
- 7 = "consist of" / "part of" expressions
- 8 = containment expressions ("contains", "holds")
- 9 = "is a" expressions

Output

Output raw CSV text with exactly this header row, followed by one row per phrase.

rule,phrase

Output nothing except the CSV text.

Identification (P0)

```
1  # Task
2  Classify the provided domain phrases into the parts of a software system (classes, attributes, associations, and similar).
3
4  ## Domain Phrases
5
6  <DOMAIN PHRASES>
7
8  ## Labels
9  Use these labels for the classification:
10
11  - C = class
12  - A = attribute
13  - V = attribute value
14  - AS = association
15  - AC = association class
16  - AG = aggregation
17  - I = inheritance
18
19  # Output
20  Output raw CSV text with exactly this header row, followed by one row per phrase.
21
22  label,element,arg1,arg2
23
24  Output nothing except the CSV text.
```

Classification (P0)

Automated Prompt Optimization

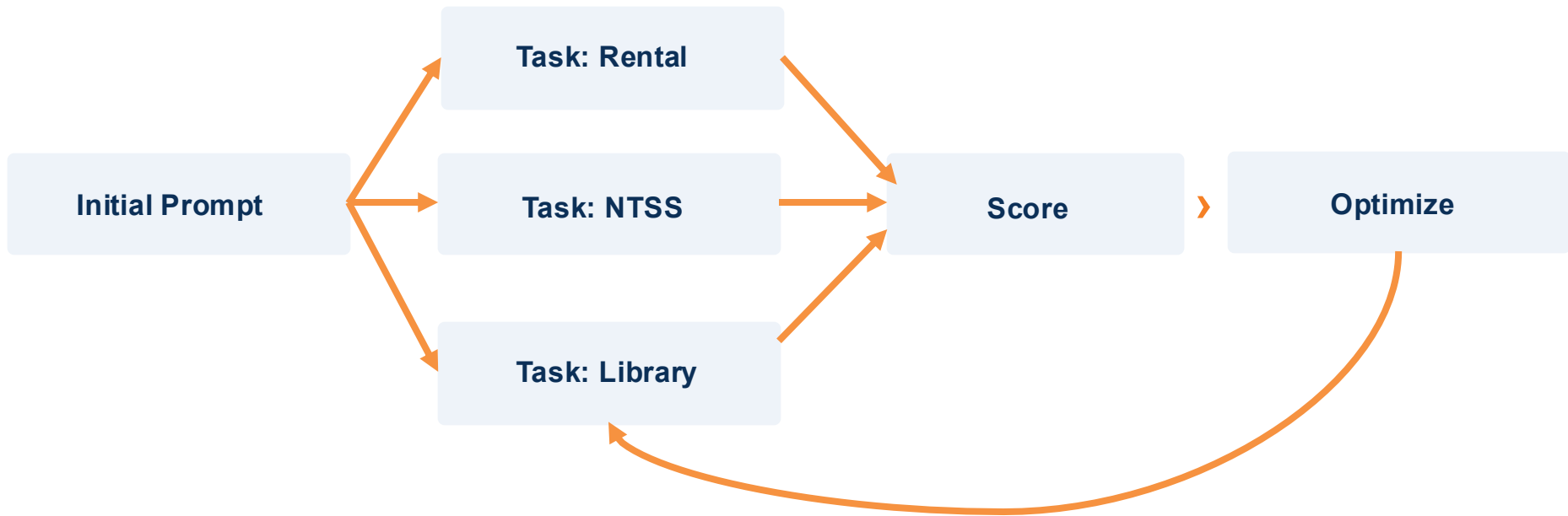
LangGraph components

We leveraged **LangGraph** as a framework for build our prompt evolution process. LangGraphs are comprised of state, nodes, and edges and together make up the components for dynamic and statefull workflows

- **LangGraph State**: Context that gets passed from node to node as the workflow executes
- **LangGraph Node**: Represents a unit of work to be completed within the graph
- **LangGraph Edge**: Serves as the highway between nodes, determining how state is passed from each node to the other

Task State Graph

Classification or Identification



Initial classification /
identification prompt to be
optimized

LLM given current prompt and
asked to either identify domain
phrases, or classify them

Take task output and score
against the ground truths

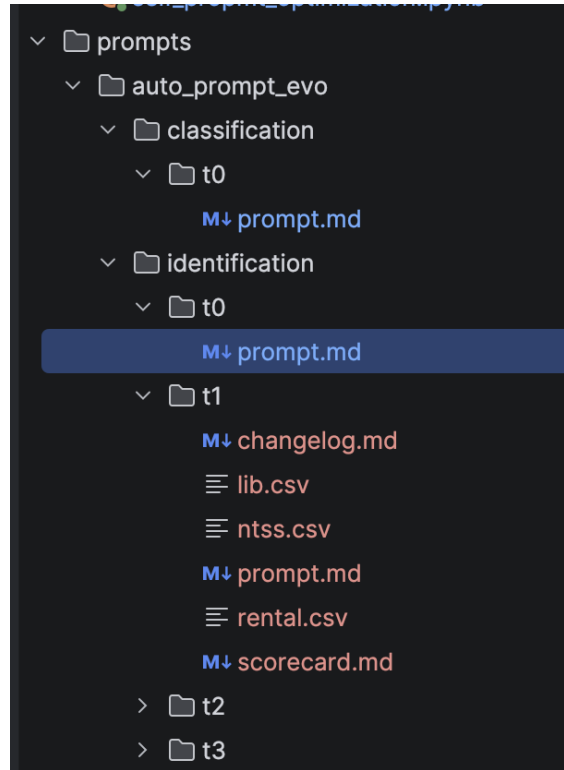
Leverage task output, ground
truth, and prompt to evaluate
ways to improve the prompt

Automated Prompt Optimization

Optimization with Lang Graph

Run Structure: Each prompt iteration includes the following:

- Optimized prompt
- Car rental task output (csv)
- NTSS task output (csv)
- Library task output (csv)
- Change log – what was changed about the prompt
- Scorecard



Automated Prompt Optimization

Prompting Strategies

None (baseline)

No prompt optimization technique applied. The model is not instructed how to improve, just to try and improve

Meta Prompting

The prompt instructs the model on how to approach or structure its response, rather than providing examples.

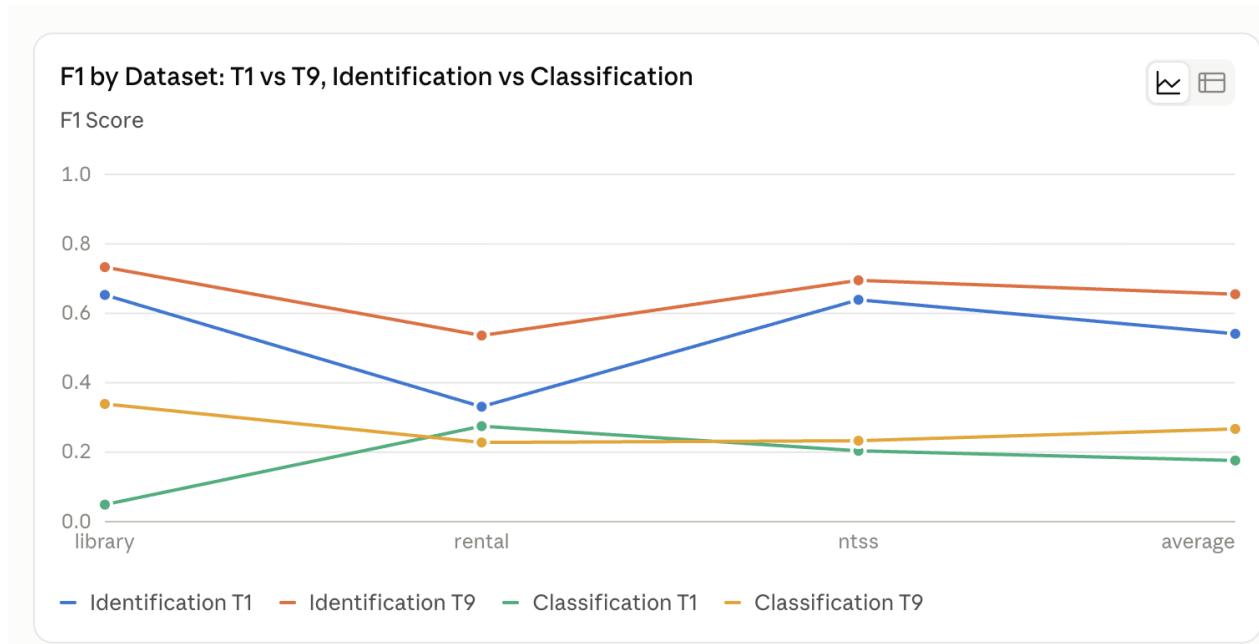
Few-Shot Prompting

The prompt includes a small set of example input/output pairs to demonstrate the desired behavior.

Results

- **Model:** Gemma 4
- Ran 10 trials for each prompt optimization strategy and baseline.
- **Identification** tasks performed significantly better in terms of F1 score with the highest being on the **NTSS domain (.749 F1)**
- **Classification** tasks struggled across each prompting strategy and domain. Best performance was using meta prompting on the **rental car domain (.4 F1)**

Prompt Optimization Strategy: None



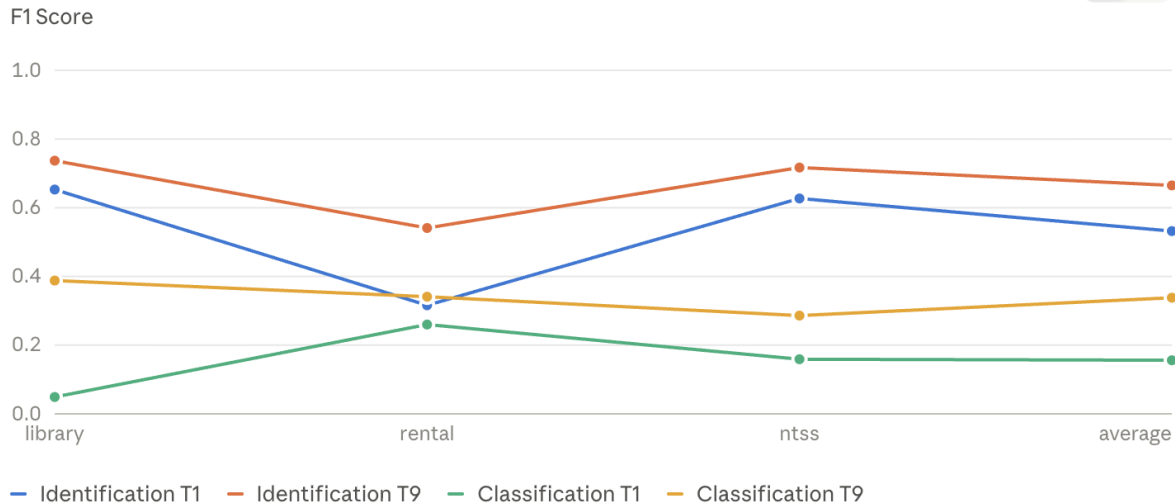
Identification performed the best across all domains

Classification showed lower F1 scores and smaller increase over 10 trials

On average 10% F1 increase across both tasks

Prompt Optimization Strategy: Few Shot

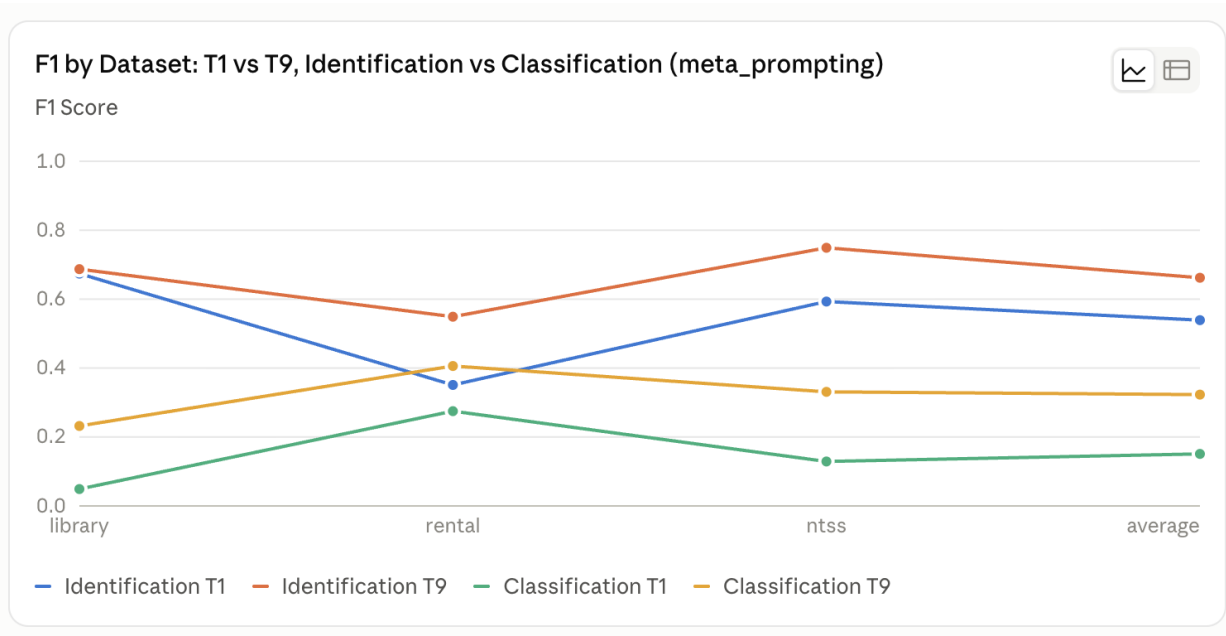
F1 by Dataset: T1 vs T9, Identification vs Classification (few_shot_prompting)



F1 for identification tasks remain high (library and NTSS domains best performers)

Classification F1 improves across 9 trials but is still relatively low

Prompt Optimization Strategy: Meta Prompting



Best performance for classification task (specificly rental)

Sharp decrease in classification F1 for library domain, but increases in others

Identification F1 remains consistently high

Testing the Evolution

Testing the Incremental Prompts Effect on different model

- **Model:** Gemma-2
- P0: Original baseline identification prompt.
- P4: Existing improved prompt developed before the Gemma 2 experiments.
- Both prompts were sent directly to Gemma 2 using the same:
 - Three domains: library, car rental, NTSS
 - 10 trials per domain
- This isolates the effect of the P0 → P4 prompt increment on Gemma 2.

Gemma 2 identification F1

Domain	P0	P4	Improvement
Library	55.94%	68.54%	+12.60 pp
Car rental	34.90%	40.96%	+6.06 pp
NTSS	52.72%	71.89%	+19.17 pp

Gemma-2 Specific Optimization: P4 → P5

- P4 responses were inspected for:
 - Missed domain phrases
 - Incorrect rule categories
 - Generic false positives
 - Missing quantities, possession clauses, lifecycle values, and “of” phrases
- P5 was created by adding targeted corrections to P4.
- The original P4 prompt and results were preserved.
- P5 then evaluated with the same setup.

Gemma 2 identification F1

Domain	P4	P5	Improvement
Library	68.54%	67.26%	-1.28 pp
Car rental	40.96%	45.20%	+4.25 pp
NTSS	71.89%	72.08%	+0.20 pp

COT Method & Results

Chain Of Thought

- COT prompting was created by combining and refining the identification and classification stages into one prompt. This was done by asking the AI to do both with the examples given by the slides as an example on both ends in the same prompt.
- COT prompting underwent its own process, where examples and rules were kept static, and only the prompting and constraints were changed per attempt.
- COT Prompting was started from a baseline given by the individual sections. However, when attempting to improve with meta prompting, results were unstable, varying wildly. As such, steps were taken back to simpler prompts that still followed knowledge gained and prompts were made and fixed in human terms.
- COT Prompting was done in Gemini Flash, to create a higher level AI for the final process, as less tokens were needed in these prompts.

Chain of Thought Optimization Process



COT Results

- Show comparisons of regular prompting vs COT prompting for each domain
- COT Prompt for all cases, case P3
- Classify the business case given in ntss-new-1 file according to the classification file and set in the format given by the brainstorm rules file. Set it up with all 9 categories as given by the brainstorm rules file. If there are any many to many classes relation, they form an association class between them, and they need any attributes necessary to make the association work. If any idea is not connected to any other idea by at least one connection, it is likely a false positive and should not be included. Only information that can stand as a quality of another or independently should be recorded at all.
- Attributes can be scalar or strings, they don't need to be specifically numbers.
- Any ideas that are appended to another are typically attributes of a that class.
- Clarify that intermediate linking actions or transaction concepts should only be extracted if they explicitly hold unique properties or independent transaction records; otherwise, express them directly as binary association relations.
- Instruct the model to strictly perform a secondary pass scanning for secondary structural nouns (e.g., sub-activities, structural facilities, or submitted materials) that host attributes or relationships, ensuring no functional entity mentioned in the text is omitted.

Car rental COT

Original vs COT

- Precision = 31.7%
 - Recall = 28.8%
 - F1 = 30.2%
- Precision = 88.89% correctness
 - Recall = 91.43% completeness
 - F1 Score = 90.14% overall score
-
- Other Scores for testing
 - 90/87/81/89/77/86

Library COT

Library original methods vs COT

- Precision = 27.0%
 - Recall = 37.0%
 - F1 Score = 31.2%
- Precision = 92.59%
 - Recall = 89.29%
 - F1 Score = 90.91%

Other Attempts F1 scores = 90/83/85

NTSS COT

Original vs COT

- Precision = 22.1%
 - Recall = 28.3%
 - F1 Score = 24.8%
- Precision = 85.71%
 - Recall = 96.00%
 - F1 score = 90.57%

Takeaways

Takeaways

Results

- In our trials, Chain of Thought (COT) prompting performed the best across each domain on both identification and classification tasks
- Classification was consistently harder for models than identification.

Takeaways

Lessons learned

- Prompt optimization is deceptively complicated
- Challenges include:
 - Consistent labeling across all domain ground truths
 - Accounting for partial matches (past tense, or other word stems)
 - Telemetry to inform how to improve optimization strategy
 - Telemetry to understand why and where the model is failing (model issue vs harness issue)
 - Consistent scoring mechanism

Takeaways

Future Work

- **Refine optimization harness to include semantic matching** – Currently we're matching on a set list of stem variations for each item. Ideally we can count TP's that are semantically similar.
- **Run optimization over more trials** – For automated optimization, we only ran for 10 trials due to budgetary concerns / hardware limitations. Ideally we could run optimization for more iterations.
- **Model Consistency** – We used a combination of open weight and frontier models. To allow for more comparable results we'd run the same trials, but group them per model

Takeaways

Questions

- **What performance is good enough?** - Unless we're building something automated, we don't need identification and classification to be perfect. They can serve as starting places.
- **Hybrid model better?** - If we know classification is poor, we can still leverage models for identification and then manually perform classification. Manual (AI assisted) classification could be faster than tweaking LM outputs.
- **Investment vs Utility?** - How do we stay mindful of token spend relative to the task we're trying to accomplish. At what point are we burning too many tokens for what we're trying to do?

Questions?